

How It's All Connected — the Hosting & Analytics Front Door

Engine · TeX backend

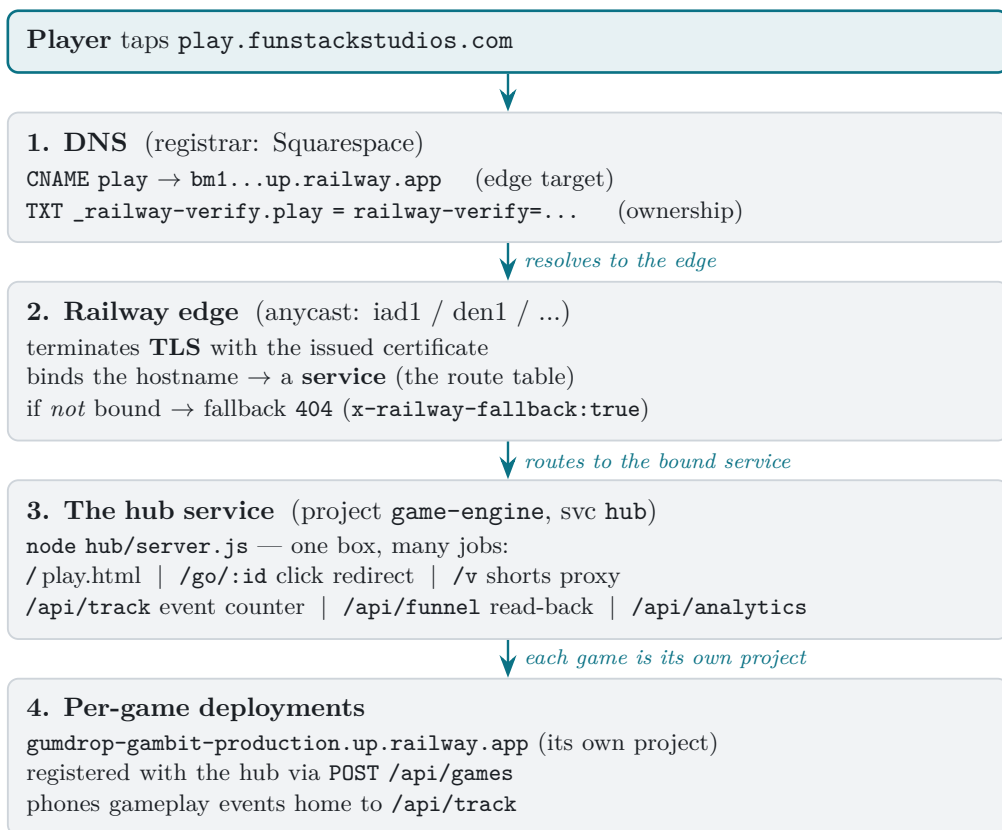
June 25, 2026

1 The one-paragraph picture

This is how a person typing `play.funstackstudios.com` on their phone ends up playing a game — and how their tap flows back to you as analytics. It is the *outside* of the engine: the registrar, Railway, the hub, and the per-game deployments. A custom domain only works when **four independent layers** all agree — DNS, the certificate, the edge route, and the app itself. Any one can be green while another is broken, so “it doesn’t work” is never enough: you check each hop. (For the *inside* — how a game is built — see `docs/ENGINE.md`.)

The trap this rig is built to catch: DNS is propagated, the cert is `VALID`, Railway’s API says the domain is fine — yet the edge still answers with a fallback 404 (`x-railway-fallback:true`) because it never bound the hostname to the service. The fix is a *redploy of the bound service*, not a DNS change. The `domain-doctor` skill automates the whole diagnosis.

2 The four layers



2.1 1 · DNS — the registrar (Squarespace)

The apex `funstackstudios.com` is registered at Squarespace, so the public hostname is configured there as custom DNS records. A Railway custom domain needs exactly two:

Type	Name	Purpose
CNAME	<code>play</code>	points the hostname at Railway's edge target
TXT	<code>_railway-verify.play</code>	proves ownership so Railway issues the cert

DNS belongs to the **owner** — these are the only steps a human does by hand. The TXT unblocks certificate issuance; once the cert is `VALID`, Railway stops requiring it. **Don't re-add the domain on Railway to "fix" things** — each re-add can mint a *new* CNAME target and force you to redo the DNS.

2.2 2 · Railway edge — TLS + the route binding

The anycast edge does two separate jobs, and *either* can be green while the other is broken:

- **Certificate** — issued once the `_railway-verify` TXT checks out. Until then you get a TLS warning, not a 404.
- **Route binding** — maps the hostname to the **hub** service. This is the layer DNS dashboards can't show. When it's stuck, the edge answers from a *fallback* app and sets `x-railway-fallback: true`. The status code lies; that header is the only tell. **The fix is a redeploy of the bound service**, which makes the edge re-read its route table — no DNS change.

2.3 3 · The hub service — one box, many jobs

A single Railway project (**game-engine**) running one service (**hub**, `node hub/server.js`). It is both the public storefront and the analytics sink:

Route	Job
<code>/</code>	<code>play.html</code> — the mobile game list; each card's thumb is the static file <code>hub/public/thumbs/<id>.jpg</code> (360×360)
<code>/go/:id</code>	logs a click (with <code>utm_source</code>) then 302-redirects to the game
<code>/v?src=</code>	proxies each game's shorts <i>same-origin</i> (byte-range mp4) so the feed plays reliably
<code>/api/track</code>	open-CORS, server-side, ad-blocker-proof event counter
<code>/api/funnel</code>	reads back clicks + events
<code>/api/analytics</code>	per-game code / asset / cost (from GitHub)

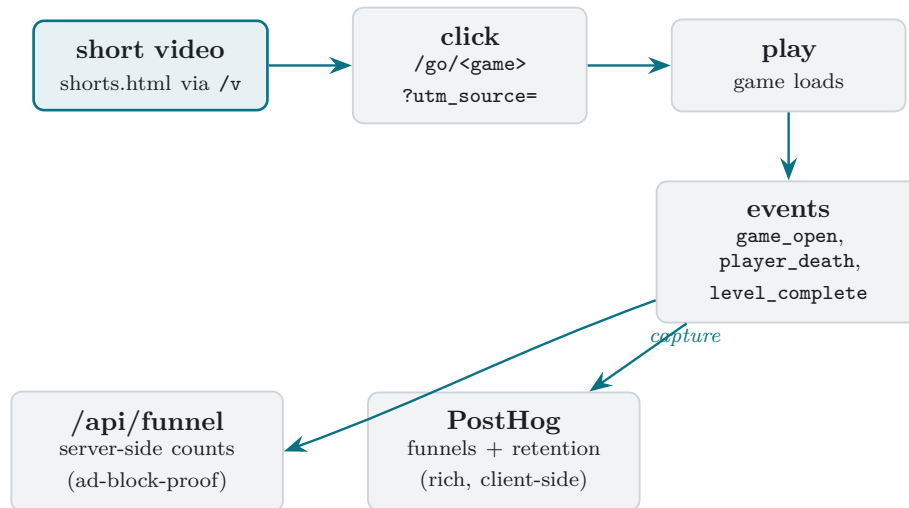
State lives in `store.js`, a tiny KV on a Railway **volume** (the games list, funnel counts), so a restart paints instantly.

2.4 4 · Per-game deployments

Each game is its *own* Railway project (e.g. `gumdrop-gambit` → `gumdrop-gambit-production.up.railway.app`), auto-deploying from its GitHub repo. A game joins the hub by calling `POST /api/games` with its `id`, `url`, and hosted `shorts` — *without* a hub code change. From then on the hub lists it, proxies its shorts, and collects its events.

3 The loop it creates — shorts → analytics

This hosting rig is what makes the flywheel measurable end to end.



- **PostHog** (public `phc_...` key) captures the rich client funnel (`$pageview` → `game_open` → `level_start` → `level_complete`, broken down by `utm_source`) and retention.
- `/api/track` → `/api/funnel` is the first-party server-side truth that survives ad-blockers (which eat ~10–40% of mobile client events). Reconcile the two to size the blind spot.
- The same gameplay events double as **eval signal** — death heatmaps, difficulty / reach calibration. See the **analytics** skill.

4 Health-checking the front door — domain-doctor

Run any time the public domain misbehaves or after a DNS change. It walks all four layers, prints a pass/fail table, and with `--fix` clears a stuck edge binding by redeploying the bound service.

```
node scripts/domain-doctor.mjs play.funstackstudios.com \  
  --project game-engine --health /health
```

#	Hop	Green means
1	DNS CNAME	hostname resolves to a Railway edge target (via DoH; <code>dig</code> isn't installed)
2	DNS TXT verify	the <code>_railway-verify</code> record is present (only needed until the cert is valid)
3	Railway cert	<code>certificateStatus == VALID</code>
4	Railway DNS	every <code>dnsRecords</code> entry is <code>PROPAGATED</code>
5	Edge serving	<code>HTTPS 200</code> <i>and</i> no <code>x-railway-fallback</code> header
6	App health	the app answers on <code>--health <path></code>

Don't skip the ladder. A correct CNAME with *no cert* = TLS error, not a 404. A valid cert with an *unbound edge* = a clean 200 from the *fallback* app, not yours. A bound edge with a *dead app* = 502. Checking the app's native `*.up.railway.app` URL isolates “app is down” from “domain isn't wired.”